Open Access

Computer Vision for Behavioral Sciences - An Introduction

LS100 — Module 01A · Video Data

Abstract

Computer vision allows digital images and videos to be transformed into structured measurements about objects, bodies, motion, and interaction. For behavioral sciences, this means that recorded video can become a source of explicit, repeatable, and scalable behavioral data. This guide introduces computer vision with a particular emphasis on how modern machine learning and deep learning models detect bodies, keypoints, and segmented regions. The main focus is on the logic of training and inference, the role of labeled data, the working principles of convolutional neural networks (CNNs), and shorter conceptual introductions to recurrent neural networks (RNNs), LSTMs, GRUs, and Vision Transformers (ViTs). The guide closes by showing how computer vision outputs become behavioral variables, and why data quality, task design, and validation matter for scientific use.

Computer vision allows digital images and videos to be transformed into structured measurements about objects, bodies, motion, and interaction. For behavioral sciences, this means that recorded video can become a source of explicit, repeatable, and scalable behavioral data. This guide introduces computer vision with a particular emphasis on how modern machine learning and deep learning models detect bodies, keypoints, and segmented regions. The main focus is on the logic of training and inference, the role of labeled data, the working principles of convolutional neural networks (CNNs), and shorter conceptual introductions to recurrent neural networks (RNNs), LSTMs, GRUs, and Vision Transformers (ViTs). The guide closes by showing how computer vision outputs become behavioral variables, and why data quality, task design, and validation matter for scientific use.

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Last updated: 2026-10-15

Authored by **Souvik Mandal, Ph.D.**

Project Leader & Instructor, Computational Behavioral Sciences, LS100, FAS, Harvard University | Linkedin ID: souvik-mandal-phd

1. Table of Contents

1. Introduction: When Video Becomes Prediction
2. What a Computer Receives from Images and Video
3. Core Tasks in Computer Vision
4. How Computer Vision Models Detect Bodies, Keypoints, and Pixels

Published Oct 23, 2026

5. What Computer Vision Can Measure in Behavioral Sciences
6. Sources of Error, Bias, and Uncertainty
7. From Computer Vision Outputs to Behavioral Variables
8. Looking Ahead

2. Introduction: From Video to Data

Digital video stores frames, pixels, and time-ordered information. Computer vision allows us to do more than store or replay videos - it detects patterns in images (and videos), identify meaningful structures, follow them across time, and turn them into structured outputs that can be interpreted scientifically.

For behavioral scientists, this changes what video can be used for. In addition to being watched manually, videos can also become a source of repeated measurements about position, posture, movement, proximity, synchrony, and change through time. In that sense, computer vision is about building systems that translate images and videos into useful quantitative data.

This guide introduces the conceptual foundations of computer vision with a particular emphasis on the part that matters most for the understanding used for research: how models are trained to detect bodies, keypoints, and pixels, and how those learned systems are used later during inference.

3. What a Computer Receives from Images and Video

At the most basic level, a computer receives images as arrays of numbers. In a grayscale image, those numbers represent brightness values. In a color image, they represent multiple channels such as red, green, and blue. A video extends this same structure through time as a sequence of such frames.

A body boundary may first appear as a change in brightness or color. Motion may first appear as differences between nearby frames. A tracked animal may first appear to an algorithm as a cluster of pixels whose location changes over time. All later interpretation depends on the quality and structure of these original measurements.

4. Core Tasks in Computer Vision

Although computer vision is a broad field, several recurring tasks are especially important for behavioral sciences:

- **Detection:** Identifying whether an object or body is present and locating it within the frame.
- **Tracking:** Following a detected object, individual, or body part across successive frames.
- **Segmentation:** Assigning pixels or regions to an object or class, often to separate a subject from the background.
- **Pose Estimation:** Estimating landmarks such as joints, limbs, or facial points to describe body configuration.

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

These tasks are conceptually distinct, but they often work together. A pose estimator may begin with a detected body. A tracker may follow a segmented region. A behavioral analysis pipeline may use detections, keypoints, and trajectories all at once.

5. How Computer Vision Models Detect Bodies, Keypoints, and Pixels

This section is the conceptual center of the guide. In modern computer vision systems, rules are not handwritten. Instead, models are trained from labeled examples to learn useful visual patterns and then apply those learned patterns to new images or videos.

5.a. Why Hand-Coded Rules Became Insufficient

Early computer vision often relied on hand-designed or hard-coded rules: thresholding brightness, subtracting a background image, detecting edges, or identifying color ranges that might correspond to a target object. In very controlled settings, these methods can still be useful. For example, a dark animal moving against a bright, uniform background may be segmented reasonably well with a carefully chosen threshold.

However, such rules become fragile as soon as the scene becomes more realistic. Lighting changes, shadows shift, bodies rotate, feathers or clothing vary in texture, multiple individuals overlap, and the background contains clutter. Under these conditions, writing fixed rules that reliably capture the target becomes extremely difficult. The rules either become too narrow and often fail, or too broad and capture wrong things.

Machine learning became attractive because it replaced many manually crafted decision rules with parameters learned from examples. Instead of explicitly telling the system every visual rule to follow, researchers provide annotated data from which the model can estimate what patterns tend to predict the desired output.

5.b. What a Machine Learning Model Is

A machine learning model is a parameterized function that maps input data to predicted outputs. In computer vision, the input may be an image, a frame, or a short video segment, and the output may be a class label, a bounding box, a set of keypoints, or a pixel-level mask.

At an abstract level, we can write this idea as:

$$\hat{y} = f(x; \theta) \tag{1}$$

where x is the input image or video data, $\hat{y}$ is the predicted output, and θ represents the model parameters. During learning, the goal is to find parameter values that make the model's predictions align well with the labeled examples.

The crucial point is that the useful rules are not typed in one by one. They are encoded in the learned parameters after exposure to many examples.

5.c. What Makes Deep Learning Different

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Deep learning is a family of machine learning methods based on multi-layer neural networks. Its power comes from representation learning. Instead of requiring the researcher to manually define all the relevant visual features, deep networks learn layered internal representations directly from data.

Earlier layers may become sensitive to local visual patterns such as edges, corners, and textures. Deeper layers may represent more complex structures such as eyes, limbs, wings, beaks, faces, or whole body configurations. This hierarchical learning is one reason deep learning has become so successful in computer vision. It can discover increasingly abstract visual patterns that support complex tasks like detection, segmentation, and pose estimation.

5.d. Convolutional Neural Networks: How Spatial Patterns Are Learned

For many years, the dominant architecture for image-based computer vision was the **convolutional neural network (CNN)**. CNNs are especially suited to images because they exploit the local and spatial structure of visual data.

A convolutional layer applies small filters, often called **kernels**, across the image. Each filter slides over local neighborhoods of pixels and produces a new map of responses called a **feature map**. Because the same filter is reused across many image locations, CNNs can efficiently learn patterns that are useful regardless of where they appear in the frame.

Several properties make CNNs powerful:

- **Local connectivity:** Each filter initially examines a small patch rather than the whole image.
- **Weight sharing:** The same filter weights are reused across positions, which reduces the number of parameters.
- **Hierarchical feature learning:** Stacked layers allow the network to move from simple local patterns to more complex and abstract structures.
- **Downsampling or pooling:** Intermediate reductions in spatial resolution help the network summarize information and enlarge the effective receptive field.

This is why CNNs became foundational for tasks such as image classification, object detection, segmentation, and pose estimation. They are particularly good at learning spatial patterns in single images or single frames.

5.e. Training: How a Model Learns from Labeled Examples

Training is the phase in which a model learns from annotated data. This is the stage where human knowledge enters the system most directly, because the model is exposed to examples that already contain the desired output labels.

Several concepts matter here:

- **Labeled examples:** Training data consists of input images or videos paired with labels such as object classes, bounding boxes, keypoint coordinates, or segmentation masks.

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

- **Parameters or weights:** These are the adjustable numerical values in the model that determine how input patterns are transformed into predictions.
- **Forward pass:** The model processes an input and produces a prediction.
- **Loss function:** The prediction is compared with the correct label, and the discrepancy is summarized as a numerical loss.
- **Optimization:** An optimization algorithm, often gradient descent or one of its variants, updates the parameters to reduce that loss.
- **Backpropagation:** The system calculates how each parameter contributed to the error and adjusts it accordingly.

This process is repeated many times across batches of examples and across multiple training passes, often called **epochs**. The goal is not to memorize a single image, but to learn parameter values that generalize well to new, unseen data.

Training also requires data partitioning. A **training set** is used to learn the parameters, a **validation set** is used to monitor tuning decisions during development, and a **test set** is used to evaluate final performance on held-out data.

5.f. Inference: Using a Trained Model on New Images or Video

Inference is the phase in which a trained model is applied to new data. No label is supplied at this stage. Instead, the model receives a new image or video frame and produces its best prediction based on what it learned during training.

This distinction between training and inference is conceptually essential. During training, the model is exposed to labeled examples and its parameters are adjusted. During inference, the parameters are fixed and the model is used as a prediction tool.

If a model is trained to identify bird species from feeder images, then inference is what happens when a new feeder image is presented and the system predicts "sparrow" or "finch." If a model is trained for pose estimation, then inference is what happens when a new frame is presented and the system predicts the likely locations of body landmarks.

5.g. How Labeled Pixels, Boxes, and Keypoints Are Used

Different computer vision tasks require different forms of labeled training data.

For **object detection**, human annotators usually draw **bounding boxes** around target objects and assign class labels such as bird, feeder, athlete, or hand. During training, the model learns to predict both object class and approximate box location. During inference, it outputs boxes, classes, and often confidence scores.

For **segmentation**, the labels are more detailed. Instead of marking only a box, annotators identify which pixels belong to the target object or class. These labels may be binary masks, instance masks, or class-specific pixel maps. During training, the model learns to assign probabilities to pixels or regions. During inference, these probabilities are converted into a predicted segmentation mask.

For **pose estimation**, annotators provide **keypoint labels** such as head, shoulder, elbow, wing joint, tail tip, or knee. Some systems are trained to predict the

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

coordinates directly, while many learn to produce **heatmaps** that indicate probable landmark locations. During inference, the model outputs estimated keypoint positions and often confidence values for each landmark.

The table below summarizes this logic.

Task	Human Label Used in Training	Typical Output at Inference	Behavioral Example
Detection	Bounding boxes and class labels	Object boxes, classes, confidence scores	Detecting when a bird is present at a feeder
Segmentation	Pixel-level masks or labeled regions	Predicted masks or per-pixel class maps	Separating an animal from a moving background
Pose Estimation	Landmark or keypoint locations	Predicted keypoints or heatmaps	Estimating body posture during movement

This is why labeled pixels matter so much. For segmentation in particular, the model does not infer masks magically. It learns from examples where humans have already indicated which pixels correspond to the target object or class.

5.h. From Single Frames to Sequences: RNNs, LSTMs, and GRUs

CNNs are especially good at analyzing spatial structure within a single image or frame. But behavioral sciences often care about events that unfold across time, such as a gesture sequence, a bout of freezing, a courtship display, or a locomotor rhythm. This is where sequence models become important.

Recurrent neural networks (RNNs) were designed to process ordered sequences by maintaining an internal state that carries information from earlier time steps to later ones. In principle, this allows a model to use temporal context rather than analyzing each frame in isolation.

However, simple RNNs struggle with long-range dependencies. **Long Short-Term Memory (LSTM)** networks and **Gated Recurrent Units (GRUs)** were developed to address this problem by using gating mechanisms that regulate what information is retained, updated, or forgotten across time.

Conceptually, the difference from CNNs is important:

- **CNNs** are primarily spatial models for learning visual structure within images.
- **RNNs, LSTMs, and GRUs** are temporal models for learning dependencies across sequences.

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

In behavioral science pipelines, CNNs may first process each frame spatially, and an LSTM or GRU may then model the evolution of those framewise features over time to classify actions or behavioral states.

5.i. Vision Transformers (ViTs) and Attention-Based Vision Models

More recent computer vision systems often use **Vision Transformers (ViTs)** or related attention-based architectures. Instead of relying on convolutional filters as their primary mechanism, ViTs divide an image into patches and use **self-attention** to model relationships among those patches.

The main conceptual advantage is that attention can capture long-range interactions more directly. A ViT can relate one part of the image to another without depending solely on many stacked local filters. This can be useful when global context matters, such as understanding how body parts relate across a whole frame or how multiple individuals are arranged relative to one another.

ViTs differ from CNNs in their basic inductive bias:

- **CNNs** assume that local spatial neighborhoods are especially important and build upward from local filters.
- **ViTs** rely more strongly on attention over image patches and often benefit from large training datasets.

In current practice, computer vision includes CNNs, transformers, and hybrid architectures, and there is no single universal model type. Different architectures emphasize different ways of learning visual structure.

5.j. Popular Models, Toolkits, and Their Trade-offs

Before comparing model names, it helps to separate three different levels of description that are often mixed together:

Level	What it means	Examples
Architecture family	The general computational design used to learn from images or sequences	Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), Gated Recurrent Units (GRUs), Vision Transformers (ViTs)
Model family	A concrete model design built within one of those architectural traditions	YOLO, Faster R-CNN, OpenPose, HRNet, DETR, ViTPose

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Level	What it means	Examples
Toolkit or workflow package	A software system that packages one or more models into an easier pipeline	MediaPipe, DeepLabCut, SLEAP

This distinction matters because some names refer to underlying model architectures, while others refer to practical software systems that help researchers use those models.

A second distinction is also important. When people talk about "speed versus accuracy," they usually mean **inference-time** trade-offs: how fast the model predicts on new images or video, and how accurate those predictions are. But a fuller comparison should also include **training-time computation**, because some models are much more expensive to train than to use.

5.k. Architecture Families and What They Are Good At

Architecture family	Main idea	Best suited for	Main limitation
CNN	Learns local spatial patterns with shared filters sliding across the image	Single-frame image tasks such as detection, segmentation, and pose estimation	Has a stronger local bias and may need many layers to capture broader context
RNN, LSTM, GRU	Learns sequential dependencies by carrying information across time steps	Temporal behavior modeling, sequence classification, frame-to-frame context	Less dominant now for image understanding itself; often replaced or supplemented by newer temporal models
ViT and transformer-based vision models	Learns relations among image patches using self-attention	Global context modeling, strong performance when pretrained well, modern high-capacity systems	Often heavier in data and compute requirements, especially during training

The main conceptual difference is that CNNs are primarily spatial models for learning visual structure within frames, RNNs, LSTMs, and GRUs are primarily temporal models for learning dependence across sequences, and ViTs are attention-based models that can relate distant image regions more directly.

5.l. Model Families by Task

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Task	Model or toolkit	Architecture basis	Custom training on new data	Custom landmarks or custom masks/ classes	Training compute	Inference compute	Typical strength	Typical trade-off
Object detection	Faster R-CNN	CNN-based, two-stage detector	Yes	Custom object classes: Yes	High	Medium to high	Strong accuracy and stable research baseline	Slower inference than real-time detectors
Object detection	SSD	CNN-based, one-stage detector	Yes	Custom object classes: Yes	Medium	Low to medium	Simpler and faster than two-stage detectors	Usually less accurate than stronger newer detectors
Object detection	YOLO	Primarily CNN-based one-stage detector family	Yes	Custom object classes: Yes; custom pose labels in pose variants: Yes	Medium	Low to medium	Strong real-time performance and broad practical adoption	Very small objects or complex scenes can still be difficult
Object detection	DETR	Transformer-based detector	Yes	Custom object classes: Yes	High	Medium to high	Global reasoning and elegant end-to-end formulation	Heavier training and often slower than lightweight detectors
Object detection	DINO	Transformer-based detector	Yes	Custom object classes: Yes	High to very high	Medium to high	Strong modern detection accuracy	Higher engineering and compute cost

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Task	Model or toolkit	Architecture basis	Custom training on new data	Custom landmarks or custom masks/ classes	Training compute	Inference compute	Typical strength	Typical trade-off
Open-vocabulary detection	Grounding DINO	Transformer-based, vision-language detector	Yes, but more advanced	Text-conditioned detection rather than fixed landmark schemas	High to very high	High	Flexible text-prompted detection	More complex to fine-tune and validate scientifically
Segmentation	U-Net	CNN encoder-decoder	Yes	Custom masks/ classes: Yes	Medium	Low to medium	Strong for dense prediction with modest data	Less powerful than larger modern foundation models in complex scenes
Segmentation	DeepLab	CNN-based segmentation family	Yes	Custom masks/ classes: Yes	Medium to high	Medium	Strong semantic segmentation with good multiscale handling	Heavier than simpler encoder-decoder models
Instance segmentation	Mask R-CNN	CNN-based two-stage detector + mask head	Yes	Custom masks/ classes: Yes	High	Medium to high	Reliable instance-level masks and boxes	Slower than lighter detectors or segmenters

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Task	Model or toolkit	Architecture basis	Custom training on new data	Custom landmarks or custom masks/ classes	Training compute	Inference compute	Typical strength	Typical trade-off
Segmentation	Mask2Former	Transformer-based segmentation	Yes	Custom masks/ classes: Yes	High to very high	High	Strong modern performance across segmentation tasks	Higher compute and engineering cost
Promptable segmentation	SAM / SAM 2	Transformer-based foundation segmentation	Adaptation possible, but not the simplest custom-training route	Custom masks can be supported through prompting or adaptation; not a landmark model	Very high if training/ adapting deeply	Medium to high	Extremely useful for interactive masking and annotation support	Not automatically the best scientific measurement model for every domain
Pose estimation	OpenPose	CNN-based, bottom-up pose estimation	Yes, but comparatively heavy	Custom keypoints: Possible	High	High	Historically influential and conceptually important	Heavy inference and older, more complex deployment
Pose estimation	HRNet	CNN-based, high-resolution architecture	Yes	Custom keypoints: Yes	High	Medium to high	Precise landmark localization	Heavier than lightweight edge models

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Task	Model or toolkit	Architecture basis	Custom training on new data	Custom landmarks or custom masks/ classes	Training compute	Inference compute	Typical strength	Typical trade-off
Pose estimation toolkit	MediaPipe with models such as BlazePose	Lightweight CNN-style deployed models inside a toolkit	Usually not the standard route for custom retraining	Custom landmarks usually no, or not straightforward in standard workflows	Low for use; custom training not the main design	Low	Very accessible, fast, and easy to deploy	Best for predefined landmark schemas, not arbitrary new anatomy definitions
Pose estimation	MoveNet	Lightweight CNN-based human pose model	Not usually treated as a flexible retraining platform	Custom landmarks No standard easy path	Low to medium	Low	Fast human pose estimation	Fixed landmark definition and less flexibility
Pose estimation	ViTPose	Transformer-based pose estimation	Yes	Custom keypoints Yes	High to very high	Medium to high	Strong modern landmark accuracy	Heavier than lightweight pose models
Pose estimation	RTMPose	Predominantly CNN-based modern pose model family	Yes	Custom keypoints Yes	Medium to high	Low to medium	Good speed-accuracy balance for pose tasks	Still requires a proper labeled dataset and engineering care

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Task	Model or toolkit	Architecture basis	Custom training on new data	Custom landmarks or custom masks/ classes	Training compute	Inference compute	Typical strength	Typical trade-off
Behavioral pose toolkit	DeepLabCut	Toolkit built around deep pose estimation models, historically CNN-heavy	Yes	Custom landmarks Yes, this is a core design goal	Medium to high	Low to medium	Widely used for custom animal and human landmarks	Performance depends strongly on labeling quality and domain coverage
Multi-animal pose toolkit	SLEAP	Toolkit using learned pose models, typically CNN-based pipelines	Yes	Custom landmarks Yes	Medium to high	Low to medium	Strong for multi-animal pose and identity-aware workflows	Setup and data design still require care
Tracking	SORT	Tracking-by-detection algorithm, not a standalone deep vision backbone	Usually uses external detector, not typically “trained” end-to-end by users	N/A	Low	Low	Simple, fast, easy to understand	Identity switches under occlusion or crowding

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Task	Model or toolkit	Architecture basis	Custom training on new data	Custom landmarks or custom masks/ classes	Training compute	Inference compute	Typical strength	Typical trade-off
Tracking	Deep SORT	Tracking-by-detection with deep appearance embeddings	External detector plus learned appearance model	N/A	Medium	Low to medium	Better identity handling than SORT	Still challenged by severe occlusion or similar-looking individuals
Tracking	ByteTrack	Tracking-by-detection association method	External detector required	N/A	Low to medium beyond detector training	Low to medium	Strong practical multi-object tracking	Quality depends heavily on detector quality
Tracking	BoT-SORT	Enhanced tracking-by-detection framework	External detector required	N/A	Low to medium beyond detector training	Medium	Strong association in harder scenes	More complex than simpler trackers
Temporal behavior modeling	CNN + LSTM or CNN + GRU	CNN for spatial features, recurrent model for temporal structure	Yes	Custom behavior labels: Yes	Medium to high	Medium	Good conceptual bridge from frame features to sequences	Can struggle with very long-range dependencies
Video action recognition	I3D, SlowFast	3D CNN-based video models	Yes	Custom behavior labels: Yes	High	High	Strong spatiotemporal modeling	Compute-heavy for training and deployment

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

Task	Model or toolkit	Architecture basis	Custom training on new data	Custom landmarks or custom masks/ classes	Training compute	Inference compute	Typical strength	Typical trade-off
Video transformers	TimeSformer, Video Swin Transformer, VideoMAE	Transformer-based video models	Yes	Custom behavior labels: Yes	Very high	High	Strong long-range temporal modeling and modern performance	Often too data- and compute-intensive for small applied projects

5.m. Detection Models

YOLO, SSD, and Faster R-CNN are all primarily **CNN-based** object detection families, but they differ in how they trade accuracy, inference speed, and computational cost.

Faster R-CNN is a two-stage detector: it first proposes candidate regions and then classifies them more carefully. This often makes it strong in accuracy, but slower at inference and heavier during training than lighter one-stage systems.

YOLO is a one-stage family designed for efficiency. It is often the practical choice when detection must run in real time or on modest hardware. It is also attractive because it is relatively open to **custom training on new classes**, and in pose variants it can be trained on **custom keypoints** as well.

DETR, DINO, and Grounding DINO are **transformer-based** detection families. They often bring stronger global reasoning, but in exchange they usually require more training compute and more engineering care. Grounding DINO is especially notable because it supports text-conditioned detection, but that flexibility does not automatically make it the best option for a tightly defined behavioral measurement task.

5.n. Segmentation Models

U-Net, DeepLab, and Mask R-CNN are largely **CNN-based** families. They remain important because they are open to **custom training on new mask annotations**, and they make the training logic particularly clear: humans label pixels or regions, and the model learns to predict those labels for unseen images.

Mask2Former and SAM / SAM 2 represent a more recent **transformer-based** direction. These models are powerful, and SAM is especially useful for annotation support and promptable mask generation. But these systems should not be confused with easy custom landmark-training platforms. They are segmentation systems, not

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

pose-estimation systems, and adapting them deeply can require substantial compute and expertise.

5.o. Pose Estimation and Landmark Models

OpenPose, HRNet, MediaPipe, and MoveNet are often mentioned together, but they do not play the same role.

OpenPose and HRNet are **CNN-based pose-estimation families**. They can be custom trained on new keypoint schemas, though that process is more demanding in some frameworks than in others.

MediaPipe is better understood as a **toolkit** rather than a single architecture. Its pose and landmark systems, such as BlazePose, are lightweight deployed models intended for efficient inference. They are extremely useful when predefined landmarks are sufficient, but they are not usually the easiest route for defining a completely new landmark set.

MoveNet is a lightweight **CNN-based** human pose model. Like MediaPipe, it is strong when the target task matches the pretrained human-landmark setup, but it is not usually treated as a flexible custom-landmark training platform.

ViTPose is **transformer-based**, while RTMPose is generally treated as a modern **CNN-based** pose family with a strong practical speed-accuracy balance. Both are open to **custom training on new keypoints**.

DeepLabCut and SLEAP are especially important because they are designed around **custom landmark training workflows**. They are not just named architectures; they are research toolkits intended to help define new keypoints, label examples, train models, and evaluate outputs in behavioral contexts.

5.p. Tracking Models

SORT, Deep SORT, ByteTrack, and BoT-SORT are not usually the first examples people think of when discussing model architecture, but they are critical in behavioral workflows involving multiple individuals. These systems usually sit **on top of detections** rather than replacing the detector itself.

That means their performance depends heavily on the quality of the upstream detector. A strong tracker cannot recover identity well if detections are unstable, missing, or inconsistent. This is why tracking should often be thought of as a downstream task built on top of detection rather than as a completely separate perception system.

5.q. Temporal Models for Behavior

CNN-based detectors or pose estimators often operate frame by frame. But many behavioral questions concern sequences rather than isolated frames. This is where RNNs, LSTMs, and GRUs become useful.

A common pattern is to use a CNN to encode each frame spatially and then use an LSTM or GRU to model how those frame-level features evolve across time. This is

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

conceptually different from a pure CNN, because the recurrent part explicitly models sequential dependence.

I3D and SlowFast are **3D CNN-based** approaches that learn spatial and temporal structure jointly from short clips. TimeSformer, Video Swin Transformer, and VideoMAE are more recent **transformer-based video models**. These systems can be powerful, but their training and inference costs are often much higher than those of simpler framewise pipelines.

5.r. A Practical Rule for Model Choice

The newest model is not automatically the best model.

A model should be chosen based on:

1. the task,
2. the annotation type available,
3. whether custom classes or landmarks are needed,
4. the available training compute,
5. the acceptable inference latency,
6. and the level of scientific validation required.

A lightweight YOLO model may be a better choice than a heavier transformer detector if the goal is real-time feeder monitoring on modest hardware. A custom DeepLabCut or SLEAP workflow may be a better choice than a fixed human-pose model if the task involves new animal landmarks. A model like SAM may be excellent for annotation support but still not be the final measurement model used in analysis.

5.s. Why Training Data Quality Matters for Behavioral Research

No model learns in the abstract. It learns from the particular images, videos, labels, species, postures, environments, and annotation practices present in the training data. For that reason, training data quality is not a minor implementation detail. It is one of the main scientific determinants of model validity.

Several problems can arise:

- **Label noise:** Bounding boxes, keypoints, or masks may be inconsistent across annotators.
- **Limited diversity:** A model trained on one species, one camera angle, or one lighting condition may not generalize to another.
- **Sampling bias:** Rare but scientifically important behaviors may be underrepresented.
- **Occlusion and visibility problems:** Some body parts may be systematically harder to label, which can distort pose estimation quality.
- **Domain shift:** Differences in background, lens distortion, compression, or recording geometry can reduce transferability.

For behavioral research, this means that model performance is never independent of the data used to train it. A model can be accurate on its original benchmark and still unreliable in a new experimental context.

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

6. What Computer Vision Can Measure in Behavioral Sciences

Once subjects, body parts, or regions can be detected reliably, computer vision outputs can be transformed into behavioral variables. A tracked position can become a trajectory. Repeated positions across time can yield speed, acceleration, turning angle, or path curvature. Keypoint coordinates can become joint angles, posture indices, gait cycles, or repeated motion sequences. Distances between individuals can become measures of proximity, spacing, synchrony, or social coordination.

This is why computer vision is so useful in behavioral sciences. It allows researchers to move from broad visual impression to explicit, repeated, and often large-scale measurement. Instead of relying only on memory or sparse manual coding, researchers can quantify how behavior unfolds across many frames, many trials, or many individuals.

7. Sources of Error, Bias, and Uncertainty

Computer vision outputs are not direct truth. They are estimates produced under specific imaging conditions, model architectures, training datasets, and inferential assumptions. Poor lighting, occlusion, motion blur, unusual camera angles, background clutter, and compression artifacts can all degrade performance. Even a well-trained model may fail when moved outside the conditions it effectively learned from.

For behavioral sciences, this means that model outputs should be treated as measurements with uncertainty rather than as infallible observations. Validation, spot checking, inter-rater comparison against human coders, and awareness of dataset limitations remain essential. A high-confidence prediction is not automatically a correct one.

8. From Computer Vision Outputs to Behavioral Variables

The most important scientific step comes after detection, segmentation, or pose estimation. Bounding boxes, masks, tracks, and keypoints are still intermediate representations. Behavioral interpretation begins when those outputs are connected to theory, experimental design, and clearly defined variables.

For example, body landmarks may be transformed into stride length, social orientation, head direction, freezing duration, or a composite behavior score. A set of trajectories may be transformed into zone occupancy, exploration rate, leader-follower dynamics, or interaction frequency. The meaning does not come from the pixels alone. It comes from how visual measurements are operationalized into behavioral constructs.

This is why computer vision is powerful but not self-sufficient. The model may supply structured visual outputs, but the scientific question determines what those outputs mean.

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/

9. Looking Ahead

This guide provides the conceptual bridge between digital video and applied behavioral analysis. The next step is to work with concrete computer vision systems and camera setups that can generate usable recordings for those workflows. In the following guide, we move from concepts to implementation by examining how to build and use a computer vision Raspberry Pi camera system.

Downloaded from: https://souvikmandal.info/teaching-learning/ls100/module-01a-video-data/